Python 90 Days Challenge

Day 10 · Loops & Iteration | GitHub Notes

Notes based on the topics covered in source PDF pages 113–124, rewritten as a concise study guide.

8.5 pass

pass is a placeholder statement. It does nothing, but it lets you keep a block syntactically valid while you plan to add code later. Unlike break and continue, it does not change the loop's flow.

Example: empty loop

```
for number in range(1, 4):
    pass
```

Example: pass inside a condition

```
for number in range(1, 4):
    if number == 2:
        pass
    print(number)
```

Output: 1, 2, 3

Statement	Effect	Typical use
break	Ends the loop	Stop after finding a value
continue	Skips to next iteration	Skip unwanted values
pass	No effect	Temporary placeholder

8.6 else with Loops

A loop's else block runs if the loop completes normally. If the loop ends because break is executed, the else block is skipped.

Normal completion

```
for number in range(1, 4):
    print(number)
else:
    print("Loop finished")
```

Output: 1, 2, 3, then Loop finished

Loop stopped by break

```
for number in range(1, 5):
    if number == 3:
        break
    print(number)
else:
    print("Loop finished")
```

Output: 1, 2. The else block does not run.

8.7 Nested Loops

A nested loop is a loop inside another loop. The inner loop completes all its iterations for each one iteration of the outer loop. Nested loops are often used for rows, columns, tables, and patterns.

```
for row in range(1, 3):
    for column in range(1, 4):
        print(row, column)
```

Output: (1, 1), (1, 2), (1, 3), (2, 1), (2, 2), (2, 3)

Simple star pattern

```
for row in range(1, 4):
    print("*" * row)
```

Output:

*

**

***

8.8 range()

range() creates a sequence of integers and is commonly used with for loops. The stop value is excluded. The optional step sets the gap or direction.

Form	Meaning	Example values
range(stop)	Starts at 0; stops before stop	range(5) → 0, 1, 2, 3, 4
range(start, stop)	Starts at start; stops before stop	range(1, 5) → 1, 2, 3, 4
range(start, stop, step)	Moves by step each time	range(2, 11, 2) → 2, 4, 6, 8, 10

Positive and negative step

```
list(range(1, 6, 1))    # 1, 2, 3, 4, 5
list(range(1, 6, 2))    # 1, 3, 5
list(range(5, 0, -1))   # 5, 4, 3, 2, 1
list(range(10, 0, -2))  # 10, 8, 6, 4, 2
```

8.9 enumerate()

enumerate() provides both the index and the value while looping. The index starts at 0 by default; use start=1 when you want counting to begin at 1.

```
word = "Python"
for index, letter in enumerate(word):
    print(index, letter)
```

Output: 0 P, 1 y, 2 t, 3 h, 4 o, 5 n

```
for index, letter in enumerate("Python", start=1):
    print(index, letter)
```

Remember: a normal loop gives each value; enumerate() gives index and value together.

8.10 zip()

zip() loops over two or more sequences together, pairing items by position. It stops when the shortest sequence runs out of items.

```
letters = "ABC"
numbers = "123"
for letter, number in zip(letters, numbers):
    print(letter, number)
```

Output: A 1, B 2, C 3

```
for letter, number in zip("ABCD", "12"):
    print(letter, number)
```

Output: A 1, B 2. The longer sequence is not fully processed.

8.11 Iterator Protocol Basics

An iterable is something you can loop over, such as a string or range. An iterator produces values one at a time. iter() creates an iterator; next() gets its next value.

Term	Meaning
Iterable	An object that can be looped over
Iterator	An object that gives values one by one
iter(value)	Creates an iterator from an iterable
next(iterator)	Returns the next value

Using iter() and next()

```
iterator = iter("ABC")
print(next(iterator))  # A
print(next(iterator))  # B
print(next(iterator))  # C
```

A for loop performs this kind of iteration internally: it gets one value at a time until there are no more values.

8.12 StopIteration

When an iterator has no more values, requesting another value raises StopIteration. A normal for loop handles the end of iteration automatically.

```
iterator = iter("AB")
print(next(iterator))  # A
print(next(iterator))  # B
# Another next(iterator) raises StopIteration
```

Infinite loop

An infinite loop never ends. A common cause is a while condition that remains True because the loop variable is not updated.

```
# Incorrect: number never changes
number = 1
while number <= 5:
    print(number)

# Correct: update number
number = 1
while number <= 5:
    print(number)
    number += 1
```

Correct output: 1, 2, 3, 4, 5

Quick Revision

Concept	Remember this
pass	Placeholder; does nothing
loop else	Runs only after normal completion, not after break
nested loops	Inner loop completes for every outer-loop iteration
range()	Stop value is excluded; step controls direction/gap
enumerate()	Returns index and value
zip()	Pairs items; stops at shortest sequence
iter() / next()	Create iterator / get next value
StopIteration	Signals that the iterator has no more values
infinite loop	Check that the while condition can become False

Study tip: Run each code example in Python and predict the output before executing it.